



Interpretable and Explainable Machine Learning

Noam Barda
MD, PhD

&

Noa Dagan
MD, PhD, MPH



Molnar, Christoph. “Interpretable machine learning. A Guide for Making Black Box Models Explainable”, 2019. <https://christophm.github.io/interpretable-ml-book/>

Motivation in general

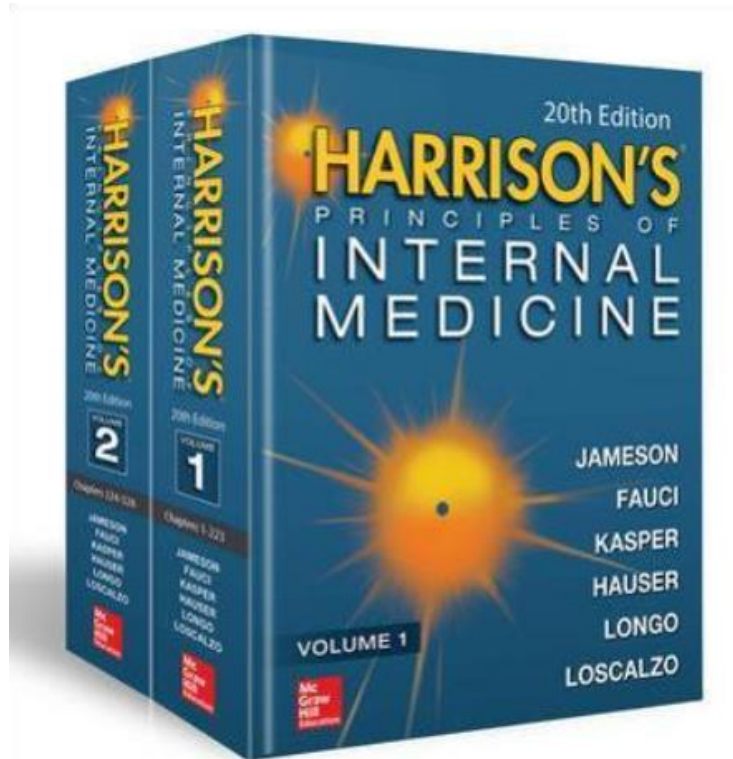
- If a machine learning model performs well, why do we not just trust the model and ignore why it made a particular decision?
- Specific metrics of accuracy are:
 - An average
 - Not enough to describe or fully grasp the model outputs
- Humans like to understand why
- Interpretable models can be debugged and audited more easily
- Interpretability is helpful for detecting bias in machine-learning models

Motivation in medicine (1)

- Medical decisions have especially high importance (high stakes)
- There is low tolerance towards wrong predictions – every single prediction is important, and guaranteed average performance is not good enough
- Bias against minority groups that are underrepresented in the data is especially not acceptable in medicine
- Clinicians are accountable for all care decisions, and they will choose to use prediction models only if they trust them

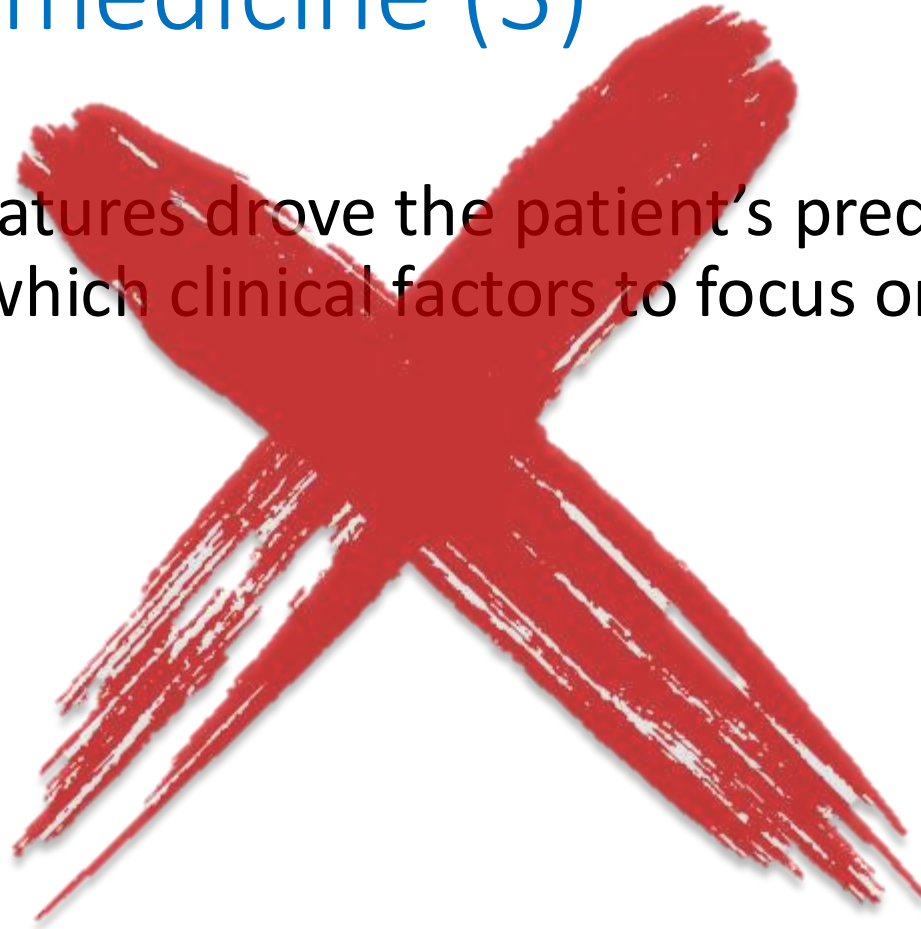
Motivation in medicine (2)

- Clinicians will choose to use prediction models only if they trust them
- What makes a clinician trust a medical machine learning model?



Motivation in medicine (3)

- If we know which features drove the patient's predicted risk to be high, we can know which clinical factors to focus on as part of the patient's care



Definitions (1)

- “Black Box Models” - models that cannot be understood by looking at their parameters (e.g., neural networks).
- “Interpretable Machine Learning” – methods and models that make the behavior and predictions of machine learning systems understandable to humans.

Definitions (2)



- **Interpretability**

- “Interpretability is the degree to which a human can (fully) understand the cause of a decision”¹
- “Interpretability is the degree to which a human can consistently predict the model’s result”²
- In other words, interpretability refers to the full transparency of why each model generates its predictions.
- However, full interpretability limits the models for specific types and sometimes requires to compromise on the model’s performance.

¹Miller, Tim. “Explanation in artificial intelligence: Insights from the social sciences.” arXiv Preprint arXiv:1706.07269. (2017).

²Kim, Been, Rajiv Khanna, and Oluwasanmi O. Koyejo. “Examples are not enough, learn to criticize! Criticism for interpretability.” Advances in Neural Information Processing Systems (2016).

Definitions (3)

- **Explainability**

- Explaining the outputs of ML models in terms that a human can understand
- That does not mean that the user can fully interpret the model's output (e.g., a doctor will be able to get some intuition of why patient A was found to be at high predicted risk but will not fully understand why patient B received a slightly lower risk)

Assumptions

- Specific features in the data are presumed to be interpretable (i.e., a human with relevant domain expertise can easily understand what they mean)

Annotations

- X – The full dataset matrix with all features
- $x^{(i)}$ – a single instance.
- x_j – the vector of a single feature for all instances
- $x_j^{(i)}$ – the value for the feature j and instance i
- y_i – the target (outcome) value for a single instance
- $\hat{f}(x^{(i)})$ or $\hat{y}$ – the predicted outcome

Taxonomy of Interpretability and Explainability Methods

- **Intrinsic vs. post hoc explainability**
 - Some models are considered interpretable due to their simple structure, such as short decision trees or sparse linear models. Post hoc explainability refers to the application of interpretation methods after model training.
- **Model-specific vs. model-agnostic explainability**
 - Model-agnostic methods are applied post hoc and are model agnostic
 - Model-specific methods can be intrinsic methods or post hoc methods that are specific to the way the model works
- **Global vs. local explainability**
 - Model level explainability vs. single instance explainability.

Human Considerations in Explainable ML

Human-friendly explanations: social aspects

- The explanations are meant for a human recipient.
- That means that we must consider social and psychological factors that may affect the way the recipient of the explanation will grasp the explanation
- In medicine, we have to consider the medical expertise of the clinicians that will receive the explanation.



Human-friendly explanations: focusing on the abnormal

- Human nature is to focus on the unexpected / abnormal.
- Humans will find causes that had a small probability but nevertheless happened as interesting events.

Human-friendly explanations: contrasting explanations

- Complete explanations are explanations that describe the contribution of each feature to the prediction. Complete explanations are hard to grasp.
- Humans usually do not ask why a certain prediction was made, but why this prediction was made instead of another prediction. In other words, **good explanations are contrastive**.
- We tend to think in counterfactual cases, i.e., “What would the prediction have been if input X had been different?”

Complete explanation vs. a contrasting explanation – an example

- Physicians might ask: “Why did my patient receive a high mortality risk?”
- A complete explanation of the physician’s question might include: The patient is a 73 years old male, who has had diabetes for 10 years, his latest sodium value was 132, his latest hemoglobin level was 16.5, and the one before that was 16.8, his weight is 73.4kg, he was admitted for 3 days 7 months ago...
- A contrastive explanation might be much simpler: In contrast to a patient with average mortality risk, this patient was admitted twice to the ICU in the last year.
- In other words, **the best explanation is the one that highlights the greatest difference between the object of interest and the reference object.**

Human-friendly explanations – applications for medical prediction models

- Even if there are multiple reasons for a specific prediction, and alternative explanations, it is best to find the most influential factors that may have tilted the patient's prediction into high/low risk.
- If the patient has abnormal laboratory values / radiographic findings / rare diseases which affected the prediction, the explanation should focus on these factors, even if alternative factors had the same amount of influence on the prediction
- We need to select explanations that would be understood / trusted by the clinician. If we find a novel risk factor that physicians have never heard about, its place is in a scientific publication rather than in the explanation of our ML model.
- Feature names should be written in the way clinicians are expected to use themselves (e.g., common medical acronyms).

Interpretable Models

General Linear Models

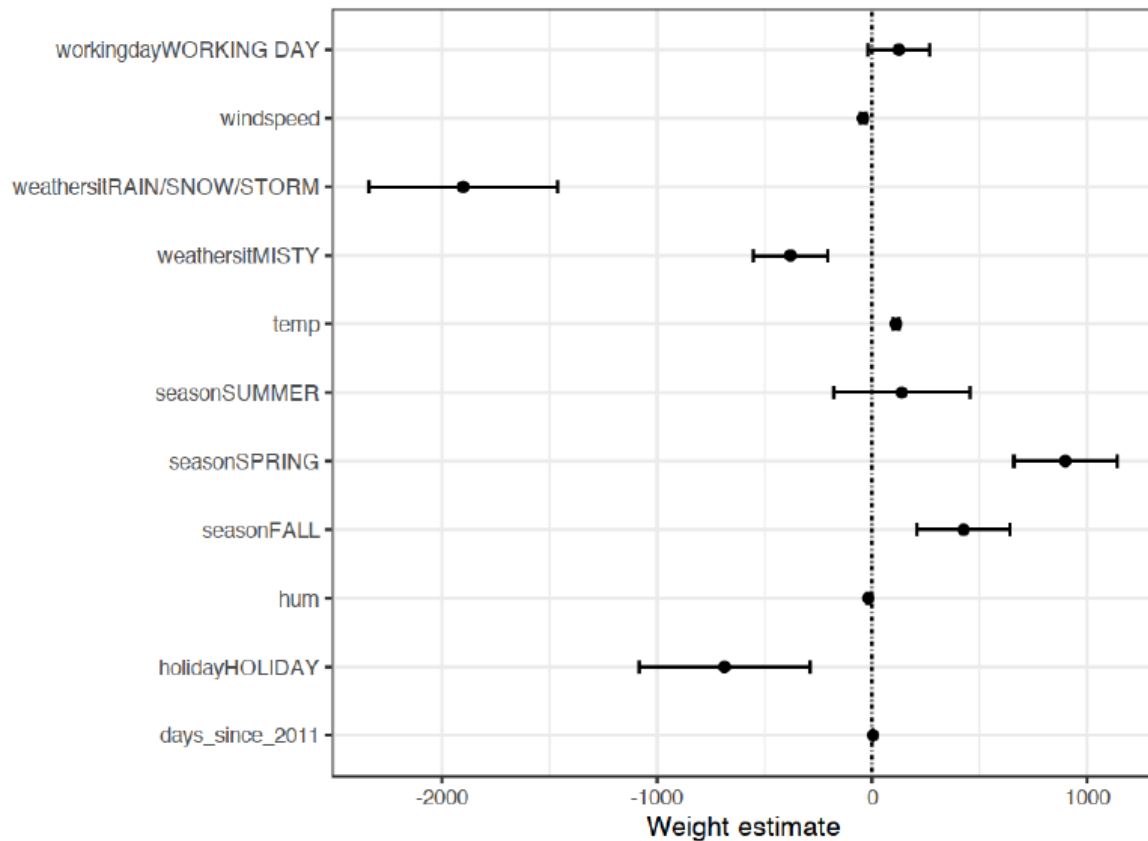
- **The classic example of interpretable models**
- A model is linear if the association between the features and the target is modeled linearly
- Monotonicity: the relationship between a feature and the target outcome always goes in the same direction over the entire range of the feature (an increase in the feature value either always leads to an increase or always to a decrease in the target outcome).
Monotonicity is useful for the interpretation of a model because it makes it easier to understand a relationship.
- Interactions: can improve model performance, but damage explainability
- Multicollinearity: strongly correlated features. Problematic in view of explainability.

Linear regression explainability (1)

- Feature interpretation is straight forward.
- $Y = 3(\text{age}) + 14(\text{male?}) + (-2)(\text{red hair})$
- For numeric features, the β coefficient represents the absolute change in outcome per one unit increase in the value of the feature ($x_j^{(i)}$)
- For categorical features, β coefficient represents the absolute change in outcome when the instance belongs to a specific category within the feature compared to the reference category

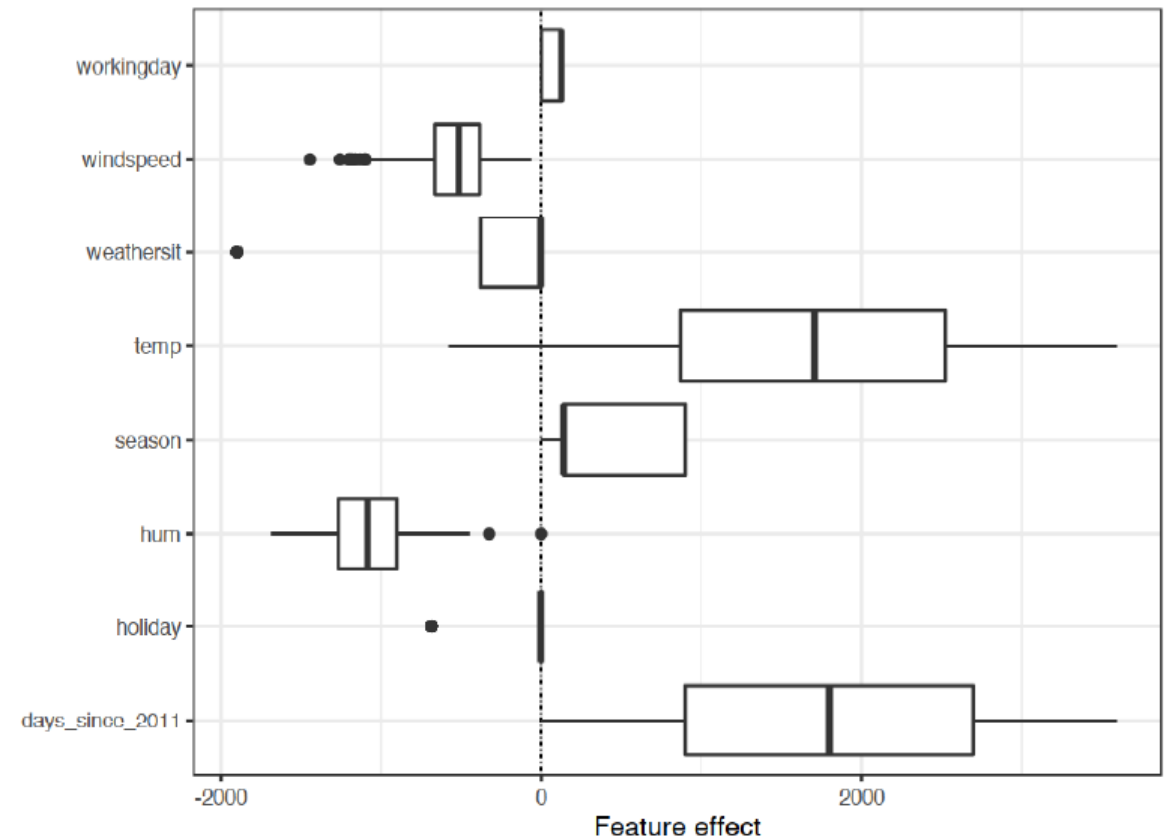
Linear regression explainability (2)

Weight Plot



Weights are displayed as points and the 95% confidence intervals as lines.

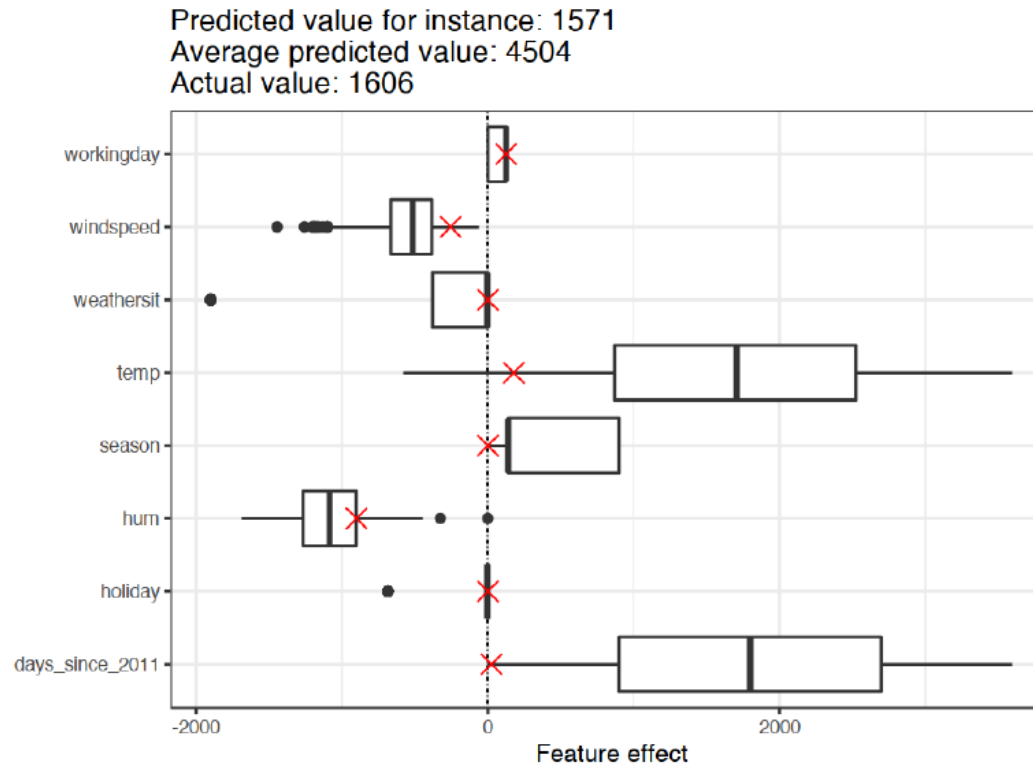
Effect Plot: $\text{effect}_j^{(i)} = w_j x_j^{(i)}$



The feature effect plot shows the distribution of effects (=feature value times feature weight) across the data per feature.

Linear regression – local explainability (1)

- Effect Plot for explaining individual predictions:



The effect plot for one instance shows the effect distribution and highlights the effects of the instance of interest. This allows us to compare the individual effects with the distribution of effects in the data.

Logistic regression explainability (1)

- Since the outcome in logistic regression is a probability between 0 and 1, the weights do not influence the probability linearly any longer
- Remember: $\log\left(\frac{P}{1-P}\right) = \sum_{j=1}^n \beta_j x_j$ and $p = \frac{1}{1 + \exp(-\sum_{j=1}^n \beta_j x_j)}$
- $\text{odds} = \frac{P}{1-P} = \exp(\sum_{j=1}^n \beta_j x_j)$
- $\frac{\text{odds}_{x_n+1}}{\text{Odds}} = \frac{\exp(\beta_0 + \beta_1 x_1 + \dots + \beta_n (x_n + 1))}{\exp(\beta_0 + \beta_1 x_1 + \dots + \beta_n x_n)} = \frac{\exp(\beta_n (x_n + 1))}{\exp(\beta_n x_n)} = \frac{\exp(\beta_n x_n) \exp(\beta_n)}{\exp(\beta_n x_n)}$
- $\frac{\text{odds}_{x_n+1}}{\text{Odds}} = \exp(\beta_n)$

Logistic regression explainability (2)

		Event	
		Yes	No
Exposure	Yes	a	b
	No	c	d

Understanding the Odds Ratio:

- $Odds = \frac{p}{1-p}$
- $Odds\ in\ the\ exposed = \frac{a/(a+b)}{b/(a+b)} = \frac{a}{b}$
- $Odds\ in\ the\ unexposed = \frac{c/(c+d)}{d/(c+d)} = \frac{c}{d}$
- $Odds\ Ratio = OR = \frac{Odds\ in\ the\ exposed\ group}{Odds\ in\ the\ unexposed\ group} = \frac{a}{b} / \frac{c}{d} = \frac{ad}{bc}$
- **In rare conditions (outcomes), which is usually the case in medicine, the $OR \approx RR$ (relative risk)**

Logistic regression explainability (3)

- $\frac{odds_{x_n+1}}{odds} = \exp(\beta_n)$
- This means that every change in one unit of a feature value x , should be interpreted in change of the logit.
- $\Rightarrow$ the interpretation of $\exp(\beta)$ is:
- For numeric features, $\exp(\beta)$ represents the factor by which the odds will change per each unit increase in the feature value. We can also say that: **$\exp(\beta)$ represents the OR (odds ratio) per each unit increase in the feature value.**
- For categorical features, $\exp(\beta)$ represents the factor by which the odds will change if the instance belongs to a specific category within the feature, compared to the reference category. We can also say that: **$\exp(\beta)$ represent the OR of belonging to a specific category within the feature compared to the reference category.**

Logistic regression explainability (4)

- Consider using LASSO (least absolute shrinkage and selection operator) to limit the number of features that are included in the model.
- Less features (a sparse model) → better explainability.

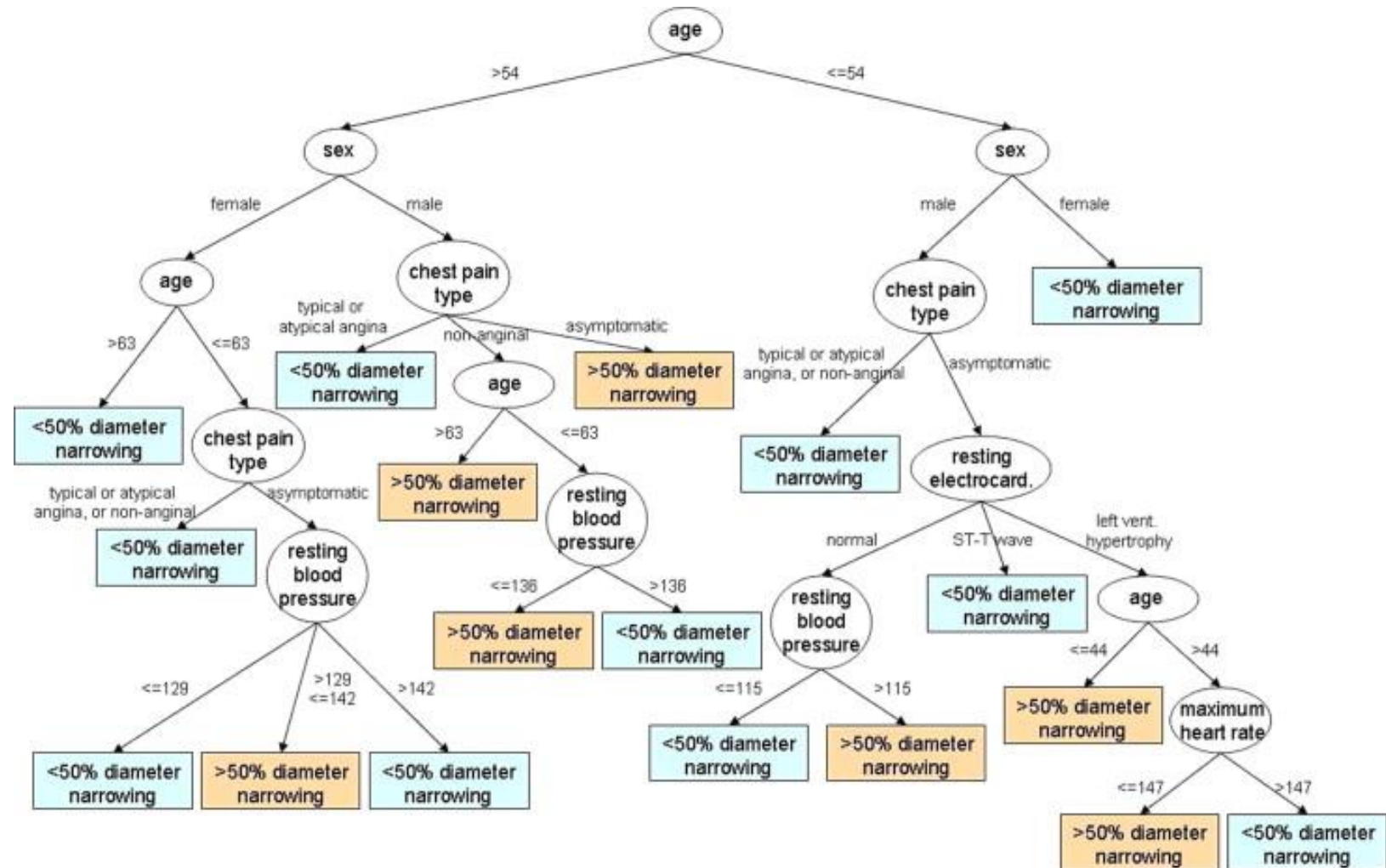
Interpretation of GLMs with interactions

- Good example of an interaction?
- If an interaction that includes our exposure of interest was added to the model, we have to describe the effect of the exposure on the outcome while considering both the coefficient of the exposure variable and the coefficients of any other relevant interaction terms.
- This is possible, but makes the interpretation more complicated and less intuitive

General Additive Models (GAMs)

- GAMs relax the restriction that the relationship must be a simple weighted sum, and instead assume that the outcome can be modeled by a sum of arbitrary functions of each feature (such as splines):
- $g(E_Y(y|x)) = \beta_0 + f_1(x_1) + f_2(x_2) + \dots + f_p(x_p)$
- These models are thus more expressive (as opposed to the tendency of GLMs to underfit), but this comes with the price of much less interpretability, because the correlation between each feature and the outcome is not a straightforward mathematical transformation.

Decision Trees



Decision Trees (1)

- Another family of interpretable models are simple decision trees.
- Algorithms for growing trees differ in the possible structure of the tree (e.g., number of splits per node), the criteria used to find the splits, when to stop splitting etc.
- The classification and regression trees (CART) algorithm is one of the most popular decision tree algorithms. CART takes a feature and determines which cut-off point minimizes:
 - The variance of y for a regression task
 - The Gini index (how “impure” a node is) of the class distribution of y for classification tasks.

$$\text{Gini} = 1 - \sum_{i=1}^k p_i^2$$

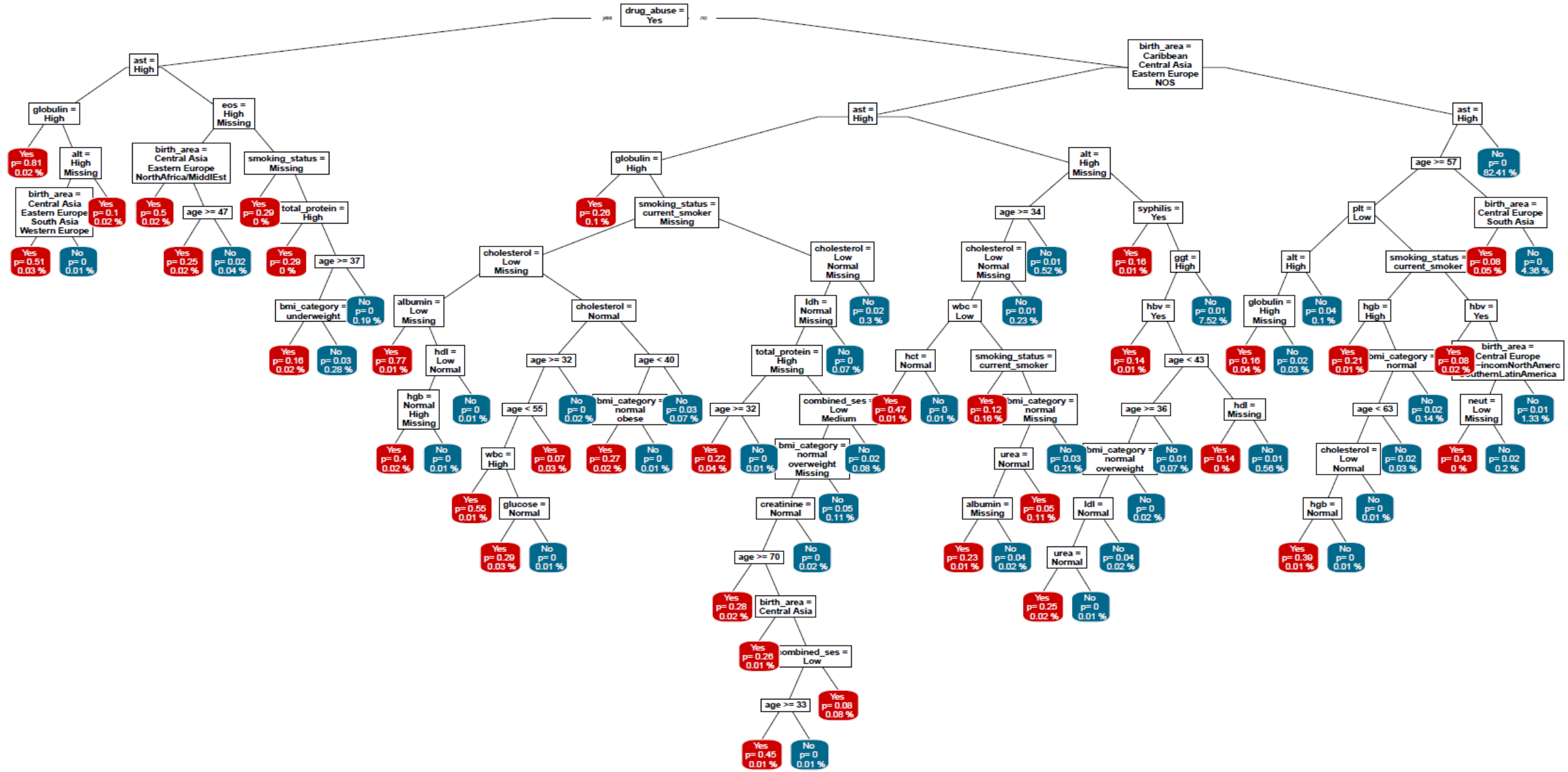
Decision Trees (2)

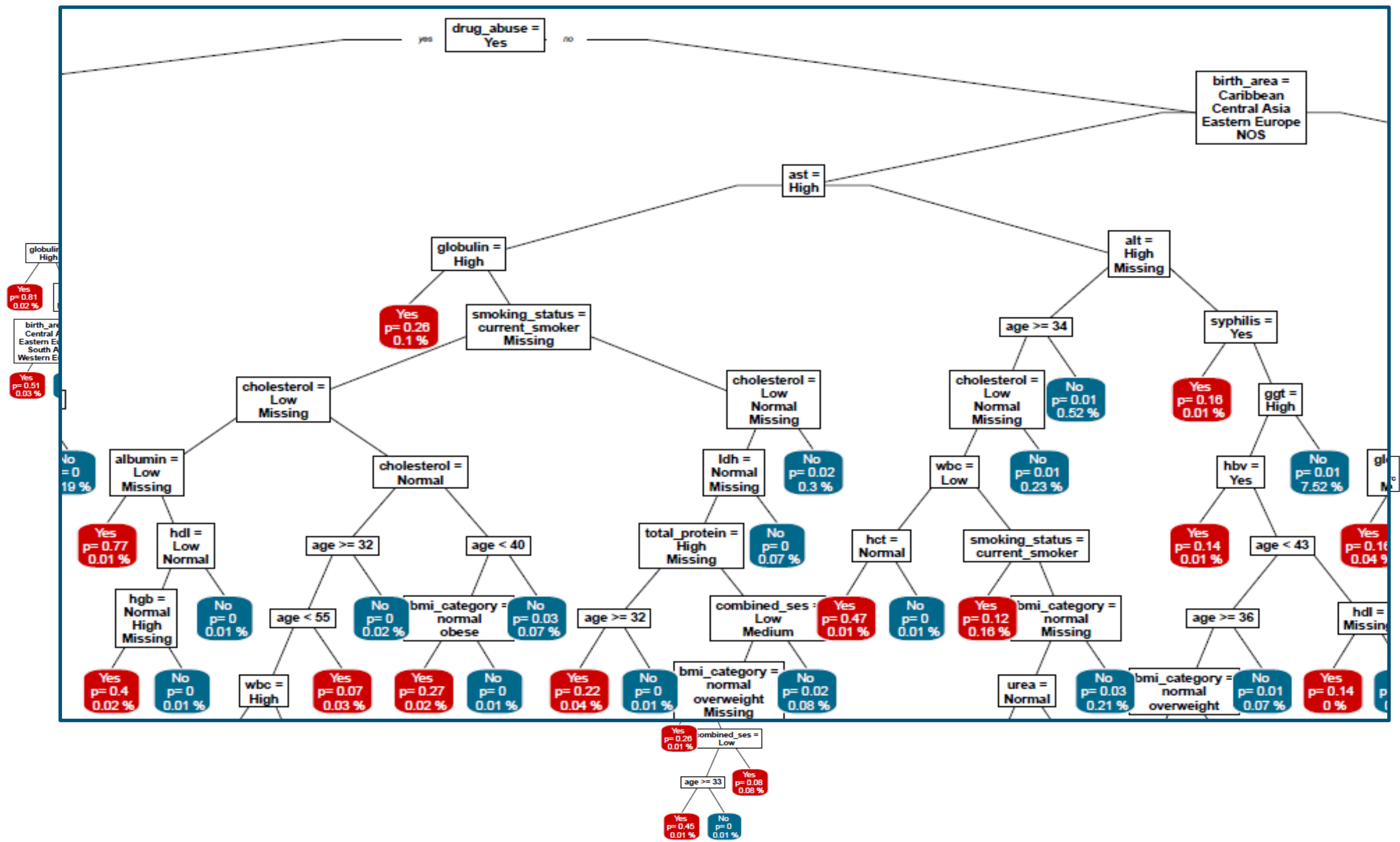
- Interpretation template: **“If feature x_1 is [smaller/bigger] than threshold c AND feature x_2 belongs to $\{A,B,C\}$ categories AND ... => then the predicted outcome is the mean value of y of the instances in that node.”**
- Global feature importance can be computed in the following way: Go through all the splits for which the feature was used and measure how much it has reduced the variance or Gini index compared to the parent node. The sum of all importances is scaled to 100. This means that each importance can be interpreted as share of the overall model importance.
- Local explainability: Individual predictions can be explained by **decomposing the decision path** into one component per feature

Decision Trees (3)

- The explainability of decision trees is also easily visualized. This visualization is also naturally contrastive, because it is easy to see what would have happened if the instance had alternative features.
- The trees naturally include interaction terms

A “white-box” algorithm example – decision tree





Model-Agnostic Methods for Explainability

Model-Agnostic Methods for Explainability

- The separation of the explainability method from the model has the advantage of flexibility, allowing the best model for the problem at hand.
- Global methods describe how features affect the prediction on average, while local methods deal with individual predictions.

Global Model-Agnostic Methods for Explainability

Global Model-Agnostic Methods for Explainability (1)

- Global methods are often expressed as expected values based on the distribution of the data.
- These methods describe average behavior.
- Useful in order to:
 - Understand the model
 - Gain potential users' trust
 - Debug the model

Partial Dependence Plot (PDP) (1)

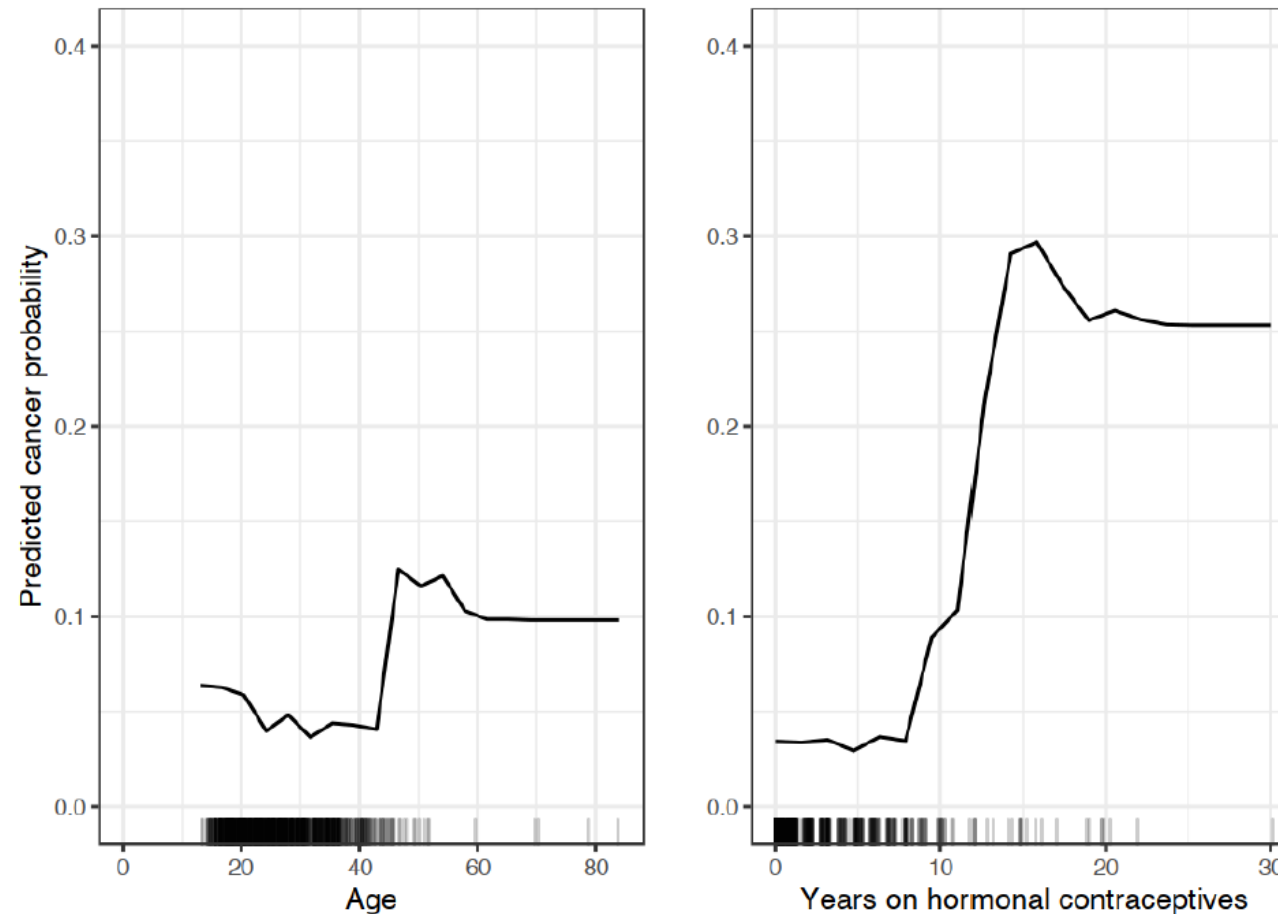
- A plot that presents the marginal effect one or two features have on the predicted outcome of ML models
- The plot is computed by calculating the average predictions over the dataset for each value of the feature(s) of interest:
(S is the set of 1-2 features for which we wish to explore the effect, C are the rest of the features used in the model, n is the number of instances in the dataset, and $\hat{f}$ is the ML model)

$$\hat{f}_S(x_S) = \frac{1}{n} \sum_{i=1}^n \hat{f}(x_S, x_C^{(i)})$$

- If S includes a numerical feature, then the X-axis in the plot can be continuous. PDPs can show whether the relationship between the target and a feature is linear, monotonic or more complex.
- If S includes a categorical variable, then the plot will take a bar chart form (for every level of S).

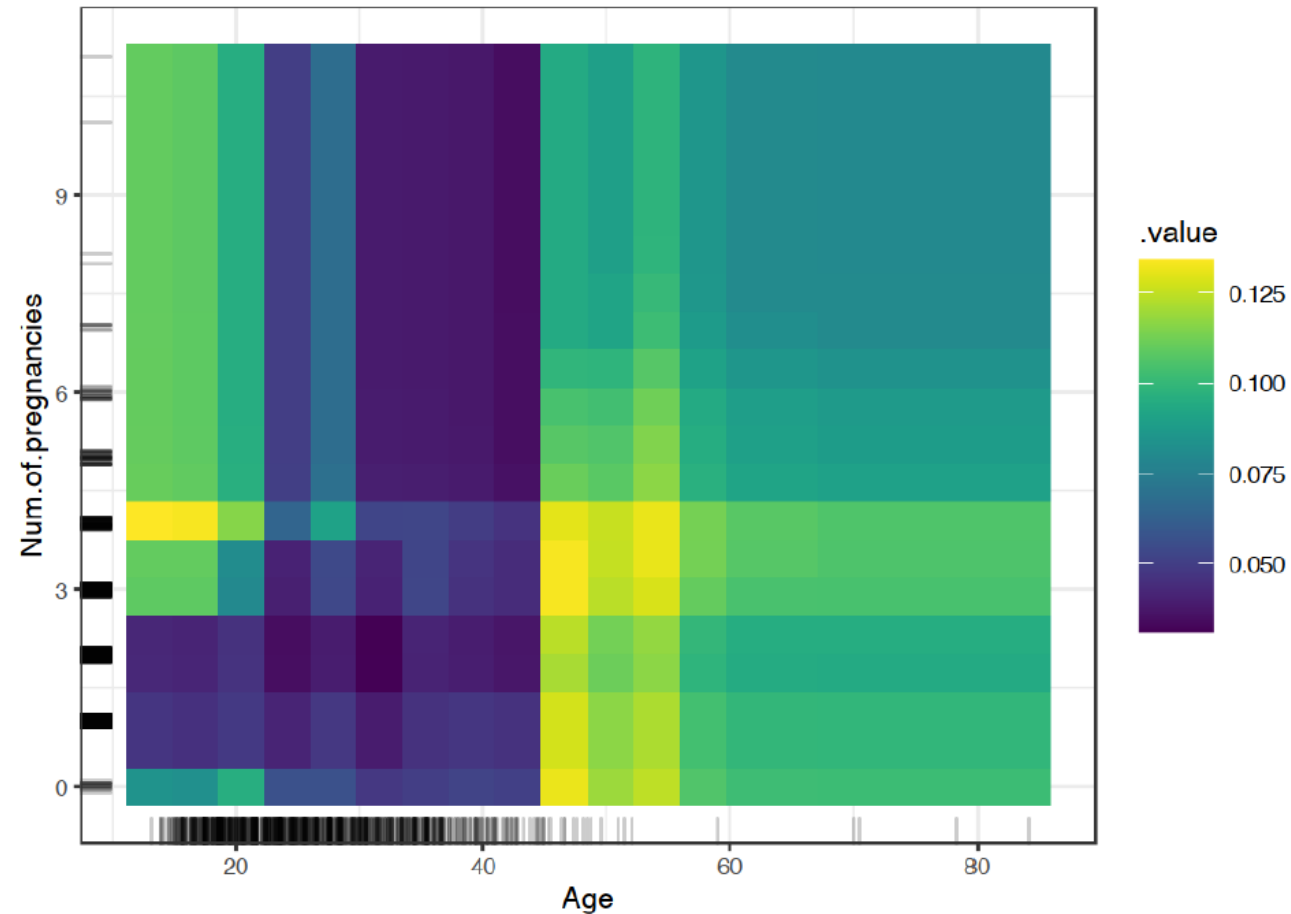
Partial Dependence Plot (PDP) (2)

- Example for single features:



Partial Dependence Plot (PDP) (3)

- Example for 2 features:



Partial Dependence Plot (PDP) (4)

- Advantages:
 - Very intuitive, and readily understood by lay people
- Disadvantages:
 - Each plot can be used to explore only 1-2 features (otherwise not plottable)
 - Assumes that the features in C are not correlated with the features in S – each value in C is evaluated for the entire dataset, even though specific values in C may not be relevant for part of the feature space
 - Heterogeneous effects might be hidden because PDPs only show the average marginal effects (i.e., when there are interactions and the same feature value can shift the prediction up or down, the effects can cancel each other, and the plot may seem wrongly flat)
 - Computationally wasteful – each value in the defined grid of C is used to calculate the predictions for the entire dataset

Partial Dependence Plot (PDP) (5)

- Software:
 - There are several R packages that implement PDPs: `iml` , `pdp` or `DALEX`.
 - In Python, PDPs are built into `scikit-learn` and you can use `PDPBox`.

Permutation Feature Importance (1)

- Permutation feature importance **measures the increase in the prediction error of the model after we permuted the feature's values**, which breaks the relationship between the feature and the true outcome.
- A feature is “important” if shuffling its values increases the model error, because it means the model relied on the feature for the prediction, and vice versa.
- Implementations:
 - R packages: iml, DALEX, vip
 - Python: alibi, scikit-learn, rfpimp

Permutation Feature Importance (2)

- The permutation feature importance algorithm (preferably implemented on the test data):

Input: Trained model $\hat{f}$, feature matrix X , target vector y , error measure $L(y, \hat{f})$.

1. Estimate the original model error $e_{orig} = L(y, \hat{f}(X))$ (e.g. mean squared error)
2. For each feature $j \in \{1, \dots, p\}$ do:
 - Generate feature matrix X_{perm} by permuting feature j in the data X . This breaks the association between feature j and true outcome y .
 - Estimate error $e_{perm} = L(Y, \hat{f}(X_{perm}))$ based on the predictions of the permuted data.
 - Calculate permutation feature importance as quotient $FI_j = e_{perm}/e_{orig}$ or difference

$$FI_j = e_{perm} - e_{orig}$$

3. Sort features by descending FI.

Permutation Feature Importance (3)

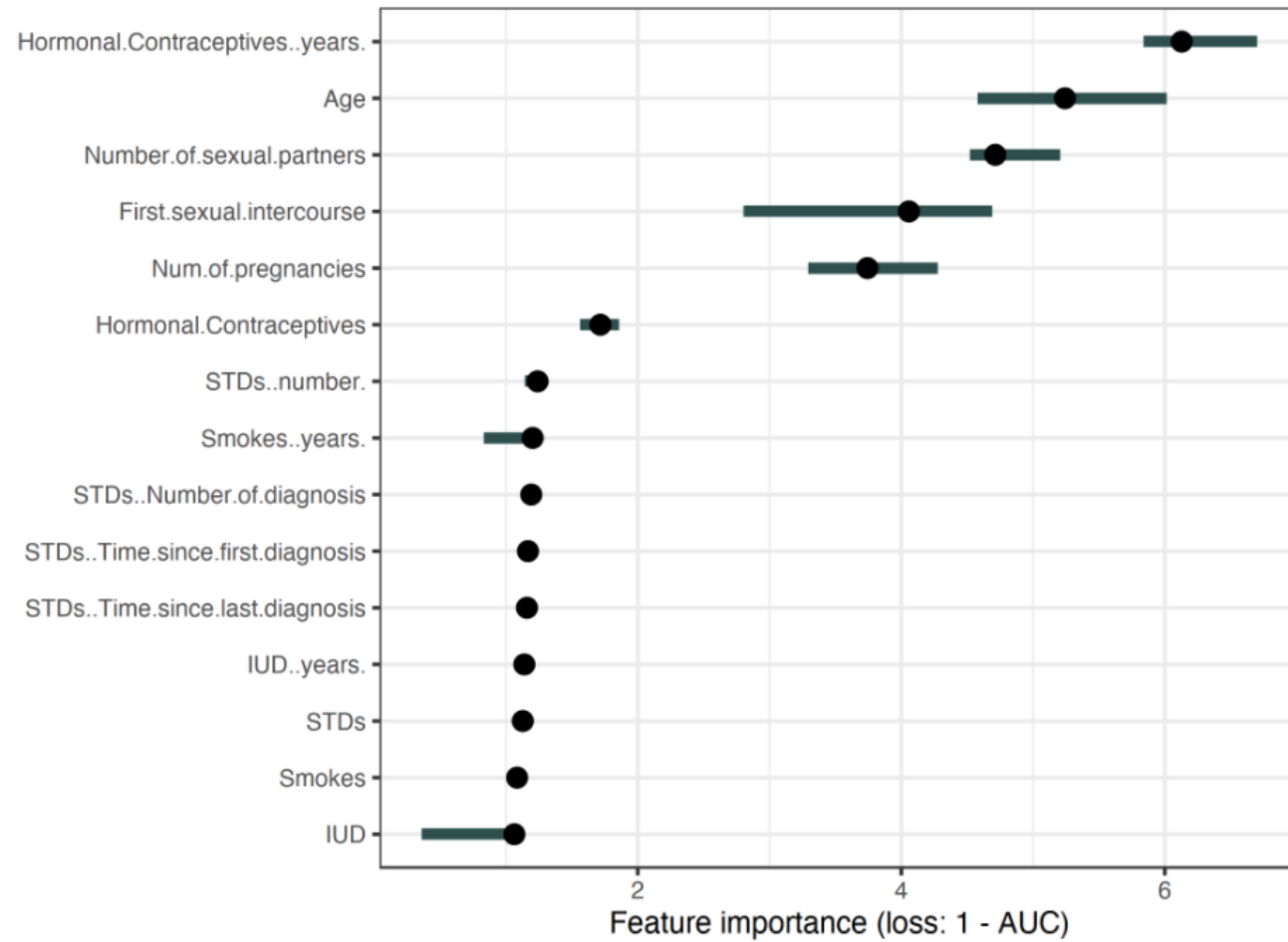


FIGURE 8.26: The importance of each of the features for predicting cervical cancer with a random forest.

The most important feature was Hormonal.Contraceptives..years.. Permuting Hormonal.Contraceptives..years. resulted in an increase in 1-AUC by a factor of 6.13

Permutation Feature Importance (4)

- Advantages:
 - Intuitive and easy to understand
 - The importance measure automatically takes into account all interactions
- Disadvantages:
 - Unstable estimate as the permutation feature importance depends on shuffling the feature, which adds randomness to the measurement. When the permutation is repeated, the results might vary greatly. Repeating the permutation and averaging the importance measures over repetitions stabilizes the measure but increases the time of computation.
 - For a model with interactions, the feature importance measures do not add up to the total drop in performance (because the importance of the interaction between two features is included in the importance measurements of both features making the overall sum larger).
 - May provide non intuitive results for a model with strongly correlated features (the shuffling may cause unrealistic data instances, which will translate to unrealistic error).
- Note:
 - The interpretation of the feature importance is with regards to the model performance (error/loss), when in some cases we might be more interested in how the features change the predicted outcome (model robustness).

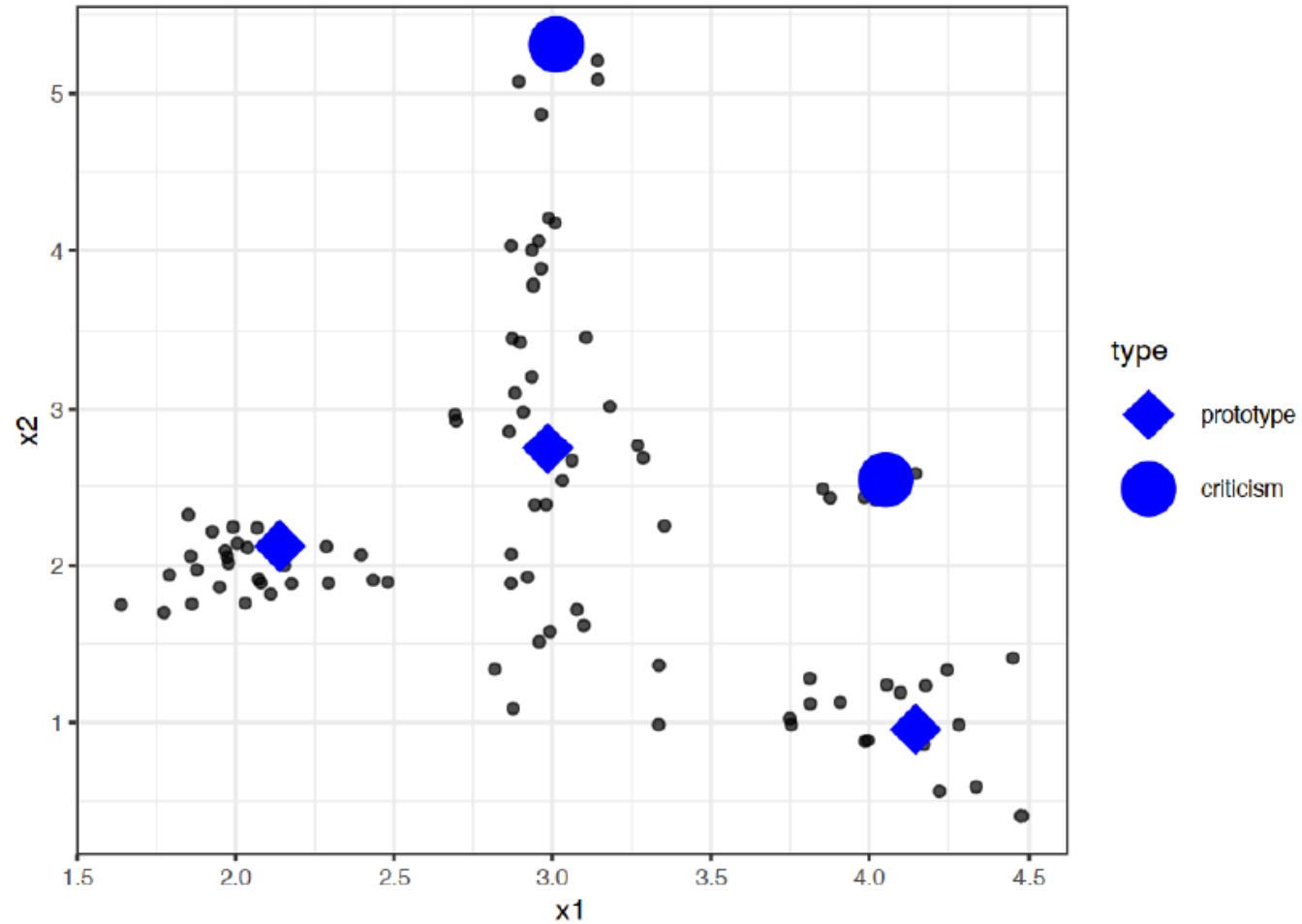
Global Surrogate Models (1)

- A global surrogate model is an interpretable model that is trained to approximate the predictions of a black box model.
- Alternative names: approximation model, metamodel, response surface model, emulator
- If the performance of the surrogate model in predicting the black-box model predictions is very good, we may sometimes prefer to use that model instead...

Prototypes and Criticisms (1)

- A prototype is a data instance that is representative of a cluster of many data instances.
- A criticism is a data instance that is not well represented by the set of prototypes. It is usually a cluster of less data points, which were not captured by the prototypes.
- There are multiple algorithms to identify good prototypes and criticisms (for example clustering algorithms related to the k-means)
- Prototypes and criticisms can be used independently from a machine learning model to describe the data, but they can also be used for interpretability

Prototypes and Criticisms (2)



Prototypes and Criticisms (3)

- The procedure for using prototypes and criticisms for interpretability:
 1. Find prototypes and criticisms
 2. Train a machine learning model as usual.
 3. Predict outcomes for the prototypes and criticisms with the machine learning model.
 4. Analyse the predictions: In which cases was the algorithm wrong? Now you have a number of examples that represent the data well and help you to find the weaknesses of the machine learning model.
- It can also be thought of as a way to identify model unfairness towards minority groups that are not predefined by protected variables.

Global Feature Importance – additional points

- A general remark regarding global feature importance:
 - These measures **mix prevalence and magnitude of the effects**.
 - **We can wrongly interoperate that features with low feature importance to be not important, when in fact they have high magnitude for a small amount of people** (say rare conditions)
- Other global feature important methods that are model specific:
 - Gini importance for random forests
 - Beta coefficients in a LR

Local Model-Agnostic Methods for Explainability

Topics

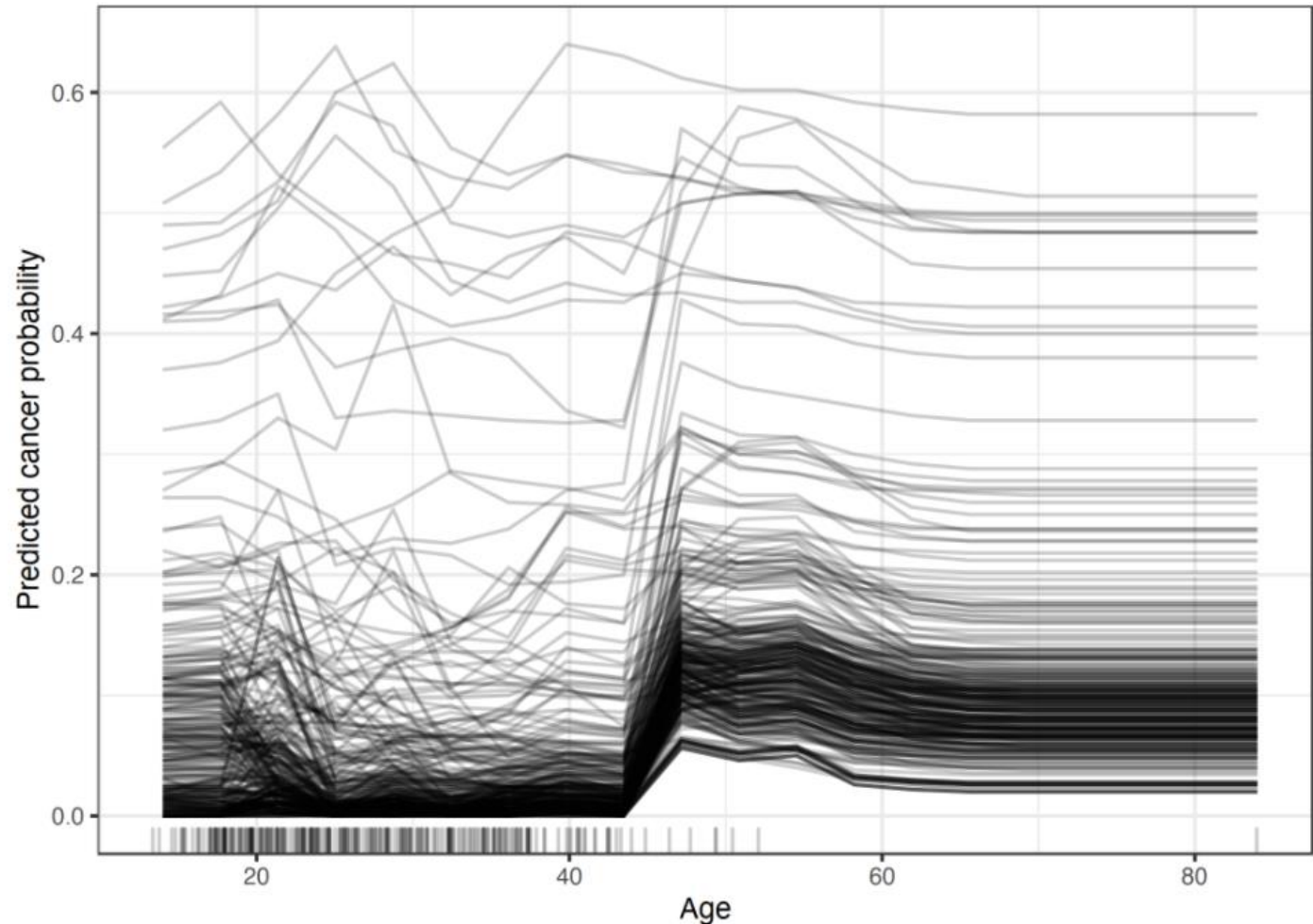
- ICE
- LIME
- Counterfactual Explanations
- Shapley Values
- SHAP

Individual Conditional Expectation (ICE) (1)

- The equivalent to a PDP for individual data instances
- An ICE plot visualizes the dependence of the prediction on a feature for each instance separately (using the vector of values for $x^{(i)}$, and each time calculating the prediction for a different value of the feature of interest in a grid of values)
- Resulting in one line per instance, compared to one line overall in partial dependence plots. In other words, **a PDP is the average of the lines of an ICE plot.**
- What is the point of looking at individual expectations instead of partial dependencies? **Partial dependence plots can obscure a heterogeneous relationship created by interactions.** In case of interactions, the ICE plot will provide much more insight.

Individual Conditional Expectation (ICE) (2)

- Example: see how the random forest prediction of each instance is associated with the feature “age”.
- In the partial dependence plot we have seen that the cancer probability increases around the age of 50, but is this true for every woman in the dataset?



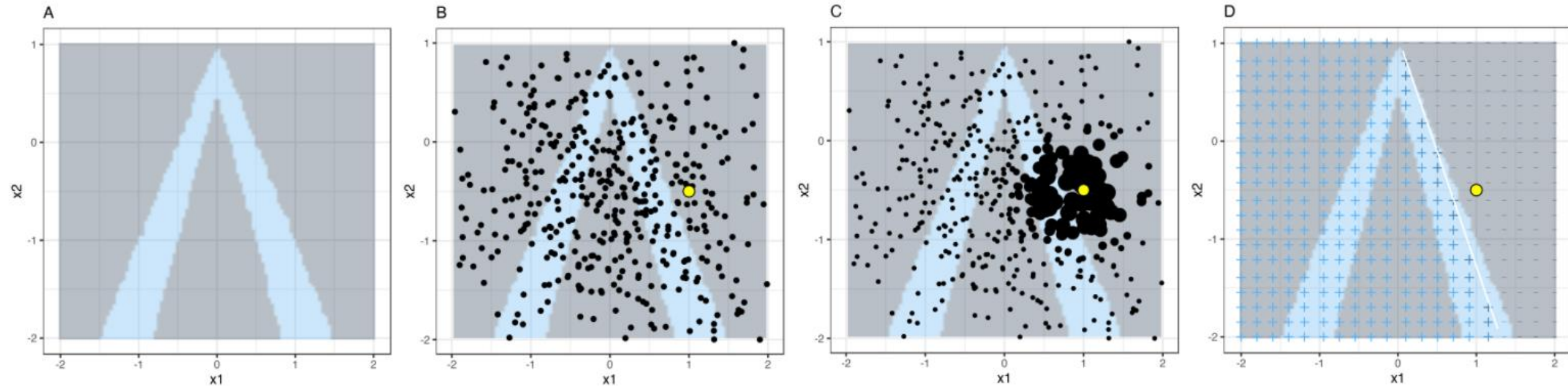
Individual Conditional Expectation (ICE) (3)

- Advantages:
 - Even more intuitive to understand
 - Unlike partial dependence plots, ICE curves can uncover heterogeneous relationships
- Disadvantages:
 - Can only display one feature meaningfully
 - Like PDPs, when one feature is correlated with another, some points in the lines might be invalid data points
 - The plot can become overcrowded, and you will not see anything. The solution is to add some transparency to the lines or to draw only a sample of the lines
- Implementations:
 - R: `iml`, `ICEbox`, `pdp`
 - Python: `scikit-learn`

Local interpretable model-agnostic explanations (LIME) (1)

- LIME is an example of using local surrogate models (interpretable models) to explain black-box predictions
- Instead of training a global surrogate model, LIME focuses on training local surrogate models to explain individual predictions
- Process:
 - Generate a new dataset consisting of perturbed samples
 - Use the BB model to get corresponding predictions for these new samples
 - Choose an instance you want to explain (x)
 - Weight the new samples according to their proximity to the instance of interest
 - Train a weighted, interpretable model on the dataset to predict the BB predictions
 - Explain the prediction by interpreting the local model

Local interpretable model-agnostic explanations (LIME) (2)



Local interpretable model-agnostic explanations (LIME) (3)

- The local surrogate model needs to be a good approximation of the BB model predictions locally, but it does not have to be a good global approximation (like in global surrogate models). This kind of accuracy is also called local fidelity.
- The local surrogate model can be expressed as the model that minimizes the following loss:
(G is the family of interpretable models such as a linear regressions, f is the BB model, g is an interpretable model, π_x is the proximity measure that defines how large the neighborhood around instance x that we consider for the explanation, $\Omega(g)$ is the regularization term to keep g simple)

$$\text{explanation}(x) = \arg \min_{g \in G} L(f, g, \pi_x) + \Omega(g)$$

Local interpretable model-agnostic explanations (LIME) (4)

- How to get the variations of the data?
 - For tabular data, perturbing each feature individually, drawing from a normal distribution with mean and standard deviation taken from the feature. This creates datapoints all over the feature space, not specific for one instance.
 - For text the solution is to turn single words on or off.
 - For images turn super-pixels (areas of approximately the same color) on or off.
- The loss function: for a linear regression, L could be for example, the sum of squared distances between the ground truth (prediction of the BB model) and the prediction of g , weighted by the distance function (measuring the distance to the sample of interest)
- The regularization term: In R and Python implementations of LIME, the regularization term, k , is the number of features we allow the linear regression to use (less => higher interpretability, more => higher fidelity).

Local interpretable model-agnostic explanations (LIME) (5)

- A non healthcare related example:

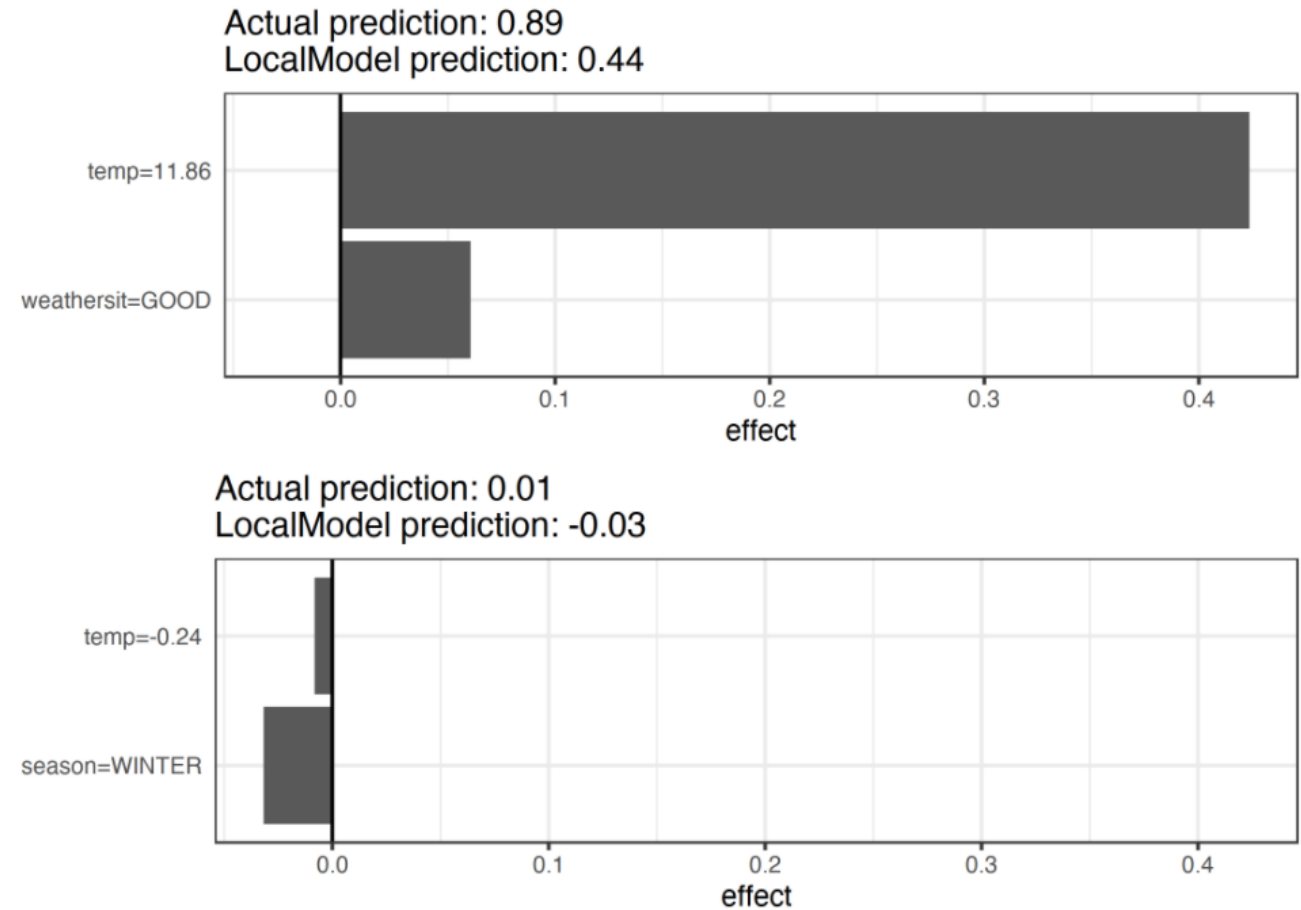


FIGURE 9.7: LIME explanations for two instances of the bike rental dataset. Warmer temperature and good weather situation have a positive effect on the prediction. The x-axis shows the feature effect. The weight times the actual feature value.

Local interpretable model-agnostic explanations (LIME) (6)

- The distance function:
 - The distance function determines the size of the instances' "neighborhood"; A small neighborhood means that an instance must be very close to influence
 - The problem: we do not have a good way to find the best size. In certain scenarios, **the explanations could be easily turned around by changing the size of the neighborhood** (for example, by creating a linear predictor in which a specific feature has a positive vs negative coefficient)

Local interpretable model-agnostic explanations (LIME) (7)

- Advantages:
 - The resulting explanations are short (= selective) and possibly contrastive => human-friendly explanations that are appropriate for lay or very busy persons
 - One of the few methods that works for tabular data, text and images
- Disadvantages:
 - Explain only some of the reasons for a prediction => less appropriate for debugging or regulatory purposes
 - Instability of the explanations:
 - The correct definition of the neighborhood is a very big, unsolved problem when using LIME with tabular data. Different choices can change the explanations. It is recommended to try different kernel settings and see what makes sense
 - The explanations of two very close points can vary greatly and the explanations for a specific instance can change when repeating the sampling process, making it difficult to trust the explanations
- Implementations:
 - R: lime, iml
 - Python: lime

Counterfactual Explanations (1)

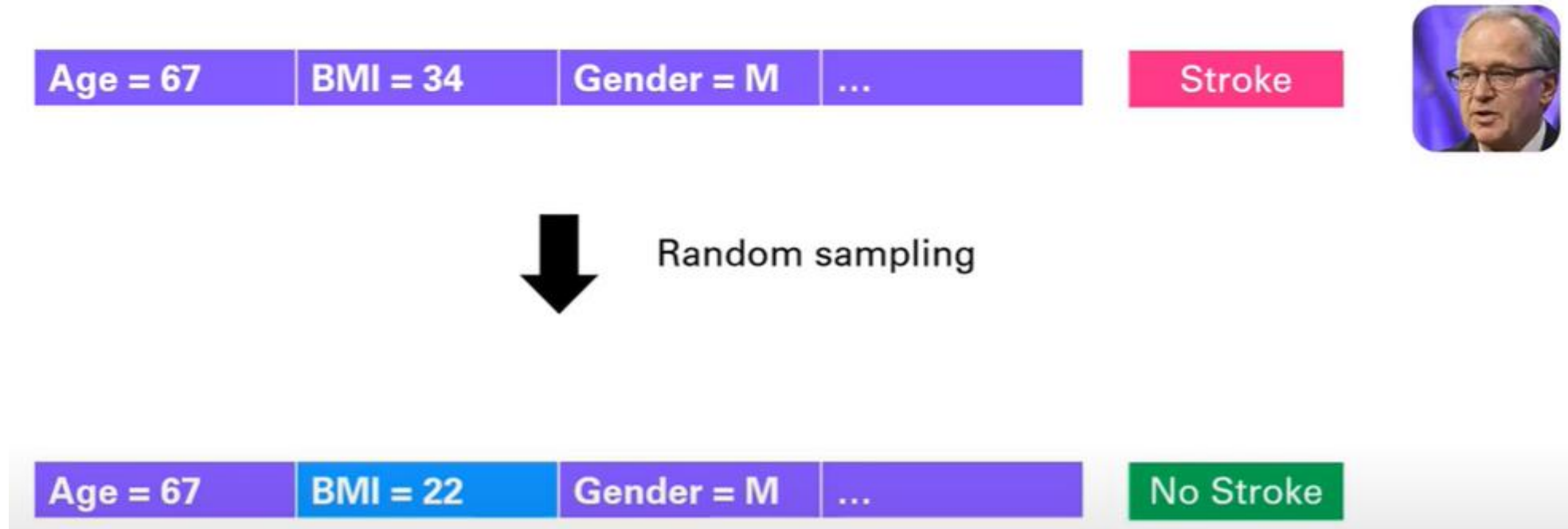
- AKA contrastive explanations
- A counterfactual explanation describes a causal situation in the form: “If X had not occurred, Y would not have occurred”. Thinking in counterfactuals requires imagining a hypothetical reality that contradicts the observed facts.
- For example: “If the patient did not have diabetes, he would not have had a heart attack”.
- In interpretable ML, counterfactual explanations can be used to explain predictions of individual instances. **Even if in reality the relationship between the features and the outcome might not be causal, we can see the features as the cause of the prediction.**

Counterfactual Explanations (2)

- We simulate counterfactuals for predictions by simply changing the feature values of an instance (creating new instances) and analyzing how the prediction changes.
- **A counterfactual explanation of a prediction describes the smallest change to the feature values that changes the prediction to a predefined output** (e.g., changing the classified output in a classification task or reaching a certain cutoff in a regression task).
- For example: “for instance X, change A and B to get a counterfactual instance”
- [This kind of minimal change required to get a different outcome is also used in adversarial attacks]

Counterfactual Explanations (3)

- An example (adopted from [this](#) recommended YouTube XAI series)



Counterfactual Explanations (4)

- A non healthcare related example – predict the probability that a customer has a good credit risk. The presented instance has a predicted probability of 24% and we ask for counterfactuals that will increase the prediction to >50%:

Data Instance:

age	sex	job	housing	savings	amount	duration	purpose
58	f	unskilled	free	little	6143	48	car

Suggested counterfactuals:

age	sex	job	amount	duration
		skilled		-20
		skilled		-24
		skilled		-22
-6		skilled		-24
-3		skilled		-24
-1		skilled		-24
-3	m			-24
-6	m			-25
-30	m	skilled		-24

Counterfactual Explanations (5)

- Multiple truths: different counterfactuals (various explanations) can explain a change in predicted outcome, and various implementations usually give several counterfactual explanations
- How to choose a good counterfactual? Usually, we cannot predetermine what would serve as a good counterfactual, and presenting several possibilities may provide more explainability
- The counterfactuals may be completely irrelevant (for example suggesting a BMI of 2.0). Current implementations thus take inputs of plausible feature values

Counterfactual Explanations (6)

- Algorithms for finding counterfactual explanations:
 - Trial and error: the naïve approach of randomly changing feature values of the instance of interest until the desired output is predicted
 - Defining a loss function and finding counterfactual explanations that minimizes this loss using an optimization algorithm. For example, the following loss was suggested by Wachter et al. (2017):

The first term is the quadratic distance between the model prediction for the counterfactual x' and the desired outcome y' (defined in advance). The second term is the distance d between the instance to be explained (x) and the counterfactual (x'). The loss minimizes both terms, and λ balances the distance in prediction (first term) against the distance in feature values (second term).

$$L(x, x', y', \lambda) = \lambda \cdot (\hat{f}(x') - y')^2 + d(x, x')$$

Counterfactual Explanations (7)

- Advantages

- Simple and easy to understand
- Straightforward – if the values are changed as suggested, it is guaranteed that the outcome will change accordingly (compared to LIME for example which is based on some local behavior and model fitting)
- Human-friendly explanations, contrastive and selective (focusing on a small number of feature changes)

- Disadvantages

- For each instance you will usually find multiple counterfactual explanations

Shapley Values (1)

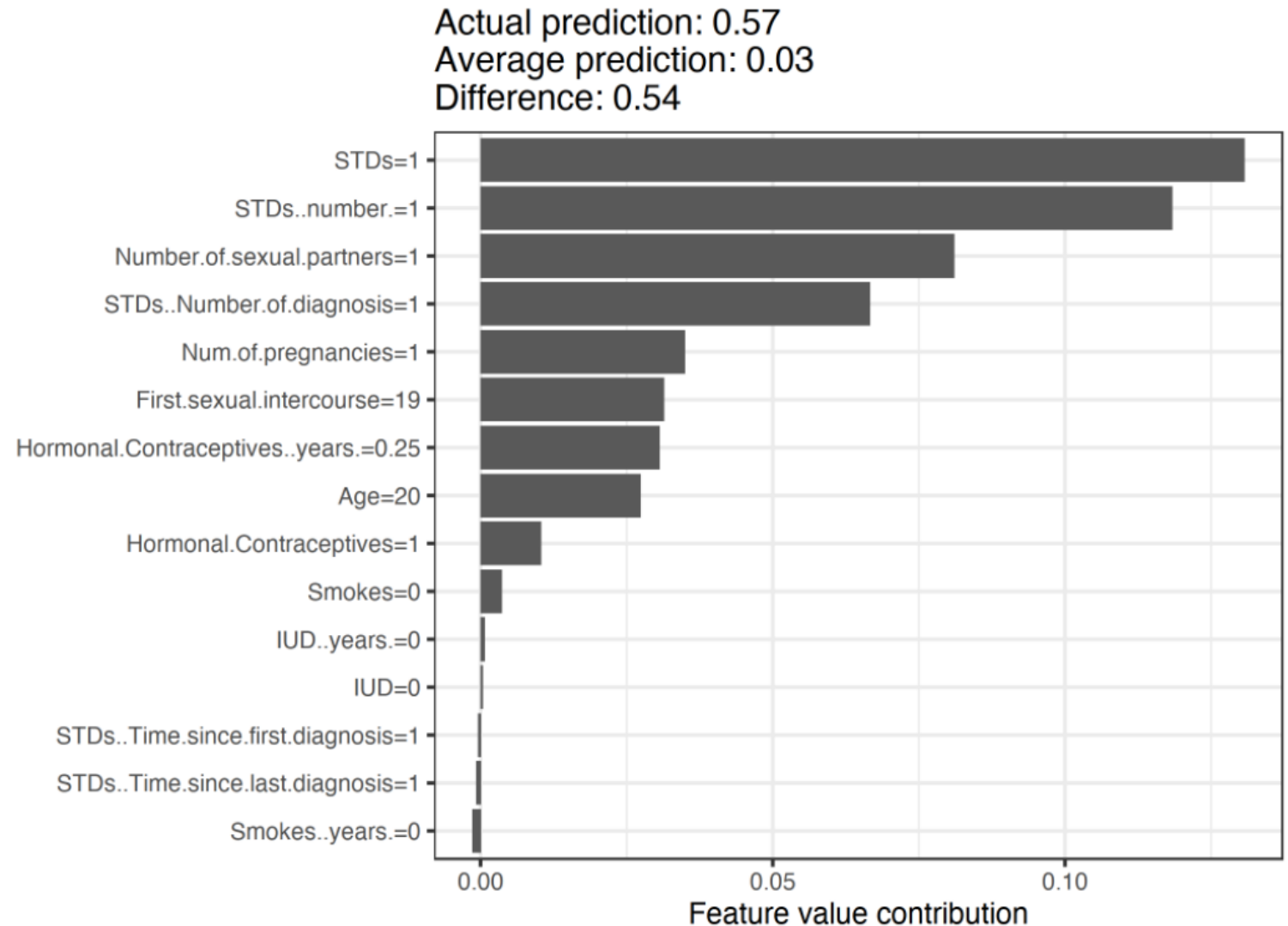
- Shapely values were defined by Lloyd Shapley in 1953 as a method for assigning fair payouts to players that cooperated in a coalition depending on their contribution to the total payout (coalitional game theory).
- A prediction can be explained by assuming that each feature value of the instance is a “player” in a game where the prediction is the payout.
- **The Shapley value is the average marginal contribution of a feature value across all possible coalitions**
 - What is the **marginal contribution**? the difference between the prediction when the value of interest is x (the original value) or x' (another random value)
 - What are all **possible coalitions**? a coalition with no other original features (in which all feature values are randomly replaced by values from the dataset), coalitions in which only a single feature in the vector remains with its original value, coalitions in which pairs of features remain with their original values, etc.

Shapley Values (2)

- **If we estimate the Shapley values for all feature values, we get the full explanation for the difference between the current prediction to the average prediction in the entire dataset** (the baseline outcome rate for a calibrated model).
- The sum of contributions yields the difference between actual and average prediction.
- Be careful to interpret the Shapley value correctly:
 - The Shapley value is “the average contribution of a feature value to the prediction across different coalitions” OR “the absolute contribution of a feature value to the difference between the actual prediction and the mean prediction given the current set of feature values”.
 - The Shapley value is NOT the difference in prediction if we were to remove the feature from the model.

Shapley Values (3)

- Example (cervical cancer prediction):
- Note that the sum of ϕ_j (the contribution of the j-th feature) is the difference between the average prediction and the specific instance prediction.



Shapley Values (4)

- The Shapely value formula:

$$\phi_j(val) = \sum_{S \subseteq \{1, \dots, p\} \setminus \{j\}} \frac{|S|! (p - |S| - 1)!}{p!} (val(S \cup \{j\}) - val(S))$$

- Where p is the number of features in the model, S is all possible subsets (coalitions) of p features excluding the feature of interest j , $val(\dots)$ is the model prediction where the values in S are being kept as the original values and the other values are randomly replaced by drawing from the dataset.
- The weight term gives higher values for very small coalitions and very large coalitions
 - Intuition for small coalitions: the feature contribution in smaller coalitions is meaningful in order to directly observed its effect in an isolated manner
 - Intuition for the importance of large coalitions: if the feature is contributing a substantial effect even though there are a lot of other features in the model then it must be important
- Each feature that is drawn randomly from the dataset is “neutralized” when considering the average effect of multiple such draws

Shapley Values (5)

- The Shapley value is the only attribution method that satisfies the following properties which together can be considered a definition of a fair payout:

Property	Explanation	
Efficiency	The feature contributions must add up to the difference of prediction for x and the average of all predictions for the dataset	$\sum_{j=1}^p \phi_j = \hat{f}(x) - E_X(\hat{f}(X))$
Symmetry	The contributions of two feature values j and k should be the same if they contribute equally to all possible coalitions	If $val(S \cup \{j\}) = val(S \cup \{k\})$ For all $S \subseteq \{1, \dots, p\} \setminus \{j, k\}$ Then $\phi_j = \phi_k$
Dummy	A feature j that does not change the predicted value – regardless of which coalition of feature values it is added to – should have a Shapley value of 0	If $val(S \cup \{j\}) = val(S)$ For all $S \subseteq \{1, \dots, p\}$ Then $\phi_j = 0$
Additivity	For a game with combined payouts $val + val^+$ the respective Shapley values can be the combination of the shapley values from each separate payout.	$\phi_j + \phi_j^+$

Shapley Values (6)

- All possible coalitions (sets) of feature values have to be evaluated with and without the j-th feature for each instance to calculate exact Shapley value.
- Problem: The computation time increases exponentially with the number of features (2^p). One solution to keep the computation time manageable is to compute contributions for only a few samples of the possible coalitions.
- An implantation that approximates the Shapely value is based on Monte-Carlo sampling (M is the number of repetitions, $\hat{f}(x_{+j}^m)$ and $\hat{f}(x_{-j}^m)$ are the prediction for x (the instance of interest), but with a random number of feature values replaced by feature values from a random data point z (another instance from the dataset), with the value of feature j (the feature of interest) from x or from z, respectively).

$$\hat{\phi}_j = \frac{1}{M} \sum_{m=1}^M \left(\hat{f}(x_{+j}^m) - \hat{f}(x_{-j}^m) \right)$$

Shapley Values (7)

Approximate Shapley estimation for single feature value:

- Output: Shapley value for the value of the j -th feature
- Required: Number of iterations M , instance of interest x , feature index j , data matrix X , and machine learning model f
 - For all $m = 1, \dots, M$:
 - Draw random instance z from the data matrix X
 - Choose a random permutation o of the feature values
 - Order instance x : $x_o = (x_{(1)}, \dots, x_{(j)}, \dots, x_{(p)})$
 - Order instance z : $z_o = (z_{(1)}, \dots, z_{(j)}, \dots, z_{(p)})$
 - Construct two new instances
 - With j : $x_{+j} = (x_{(1)}, \dots, x_{(j-1)}, x_{(j)}, z_{(j+1)}, \dots, z_{(p)})$
 - Without j : $x_{-j} = (x_{(1)}, \dots, x_{(j-1)}, z_{(j)}, z_{(j+1)}, \dots, z_{(p)})$
 - Compute marginal contribution: $\phi_j^m = \hat{f}(x_{+j}) - \hat{f}(x_{-j})$
- Compute Shapley value as the average: $\phi_j(x) = \frac{1}{M} \sum_{m=1}^M \phi_j^m$

Shapley Values (8)

- Advantages:
 - The only explanation method with a **solid theory**
 - Might be the only method to deliver a **full explanation** for each instance
 - **The difference between the prediction and the average prediction is fairly distributed among the feature values of the instance** (the Efficiency property) unlike in LIME, for example. LIME does not guarantee that the prediction is fairly distributed among the features.

Shapley Values (9)

- Disadvantages:
 - **Requires a lot of computing time**; In 99.9% of real-world problems, only the approximate solution is feasible. Decreasing M reduces computation time, but increases the variance of the Shapley value.
 - **Often misinterpreted** to be the difference of the predicted value after removing the feature from the model training
 - Explanations created with the Shapley value method always use all the features and **cannot provide sparse explanations**
 - **Not contrastive - cannot be used to make statements about changes in prediction for changes in the input** (LIME on the other hand gives a model with coefficients that can be translated to statements such as “if the BMI will increase in 5 points the risk prediction would have increased by 10%”)
 - Like many other permutation-based interpretation methods, the Shapley value method suffers from **inclusion of unrealistic data instances when features are correlated** (can be fixed if features are sampled conditional on the features that are already in the team, but then the theory and properties of the Shapley values do not hold anymore)

SHapley Additive exPlanations (SHAP) (1)

- **An alternative estimation method for Shapley values** suggested in 2017
- SHAP proposes KernelSHAP, **an alternative, kernel-based estimation approach for Shapley values, inspired by local surrogate models**
- There are many implementations of SHAP, and many presentations that translate the local SHAP values to global model explanation
- In SHAP, **the Shapley value explanation is represented as an additive feature attribution method, a linear model**. This view connects LIME and Shapley values (g is the explanation model, ϕ_0 is the average model prediction, and ϕ_j is the Shapely value for feature j):

$$g(x') = \phi_0 + \sum_{j=1}^M \phi_j$$

SHapley Additive exPlanations (SHAP) (2)

- **KernelSHAP is a way to estimate all Shapely values for an instance x using a weighted linear regression (unlike the Shapley algorithms that evaluate the Shapely value separately for each feature)**
- There are other, specific types of SHAP implementations, like TreeSAHP or DeepSHAP, but KernelSHAP is the only type that is model agnostic.
- KernelSHAP consists of 5 steps:
 - Sample coalitions $z'_k \in \{0, 1\}^M$, $k \in \{1, \dots, K\}$ (1 = feature present in coalition, 0 = feature absent).
 - Get prediction for each z'_k by first converting z'_k to the original feature space and then applying model $\hat{f} : \hat{f}(h_x(z'_k))$
 - Compute the weight for each z'_k with the SHAP kernel.
 - Fit weighted linear model.
 - Return Shapley values ϕ_k , the coefficients from the linear model.

SHapley Additive exPlanations (SHAP) (3)

- **Sampling the coalitions:** randomly sampling K coalitions (vectors of 1s and 0s). Instead of a completely random process, it is more efficient to sample coalitions with many features and coalitions with very small number of features, because these will get higher weights.
- **Converting the coalition to the original feature space:** done by the function $h_x(z'_k)$ by creating a feature vector in which the 1 positions are taken from the features of the instance of interest, and the 0 positions are taken from another random instance

SHapley Additive exPlanations (SHAP) (4)

- **The weighting:** SHAP authors suggest the following weight for each new sample in order to achieve Shapley compliant weighting:
(M is the maximal number of features and $|Z'|$ is the current size of the chosen coalition)

$$\pi_x(z') = \frac{(M - 1)}{\binom{M}{|z'|} |z'| (M - |z'|)}$$

- **The linear regression model:**

$$g(z') = \phi_0 + \sum_{j=1}^M \phi_j z'_j$$

- **The Loss function** – a simple sum of squared errors with the defined weights:

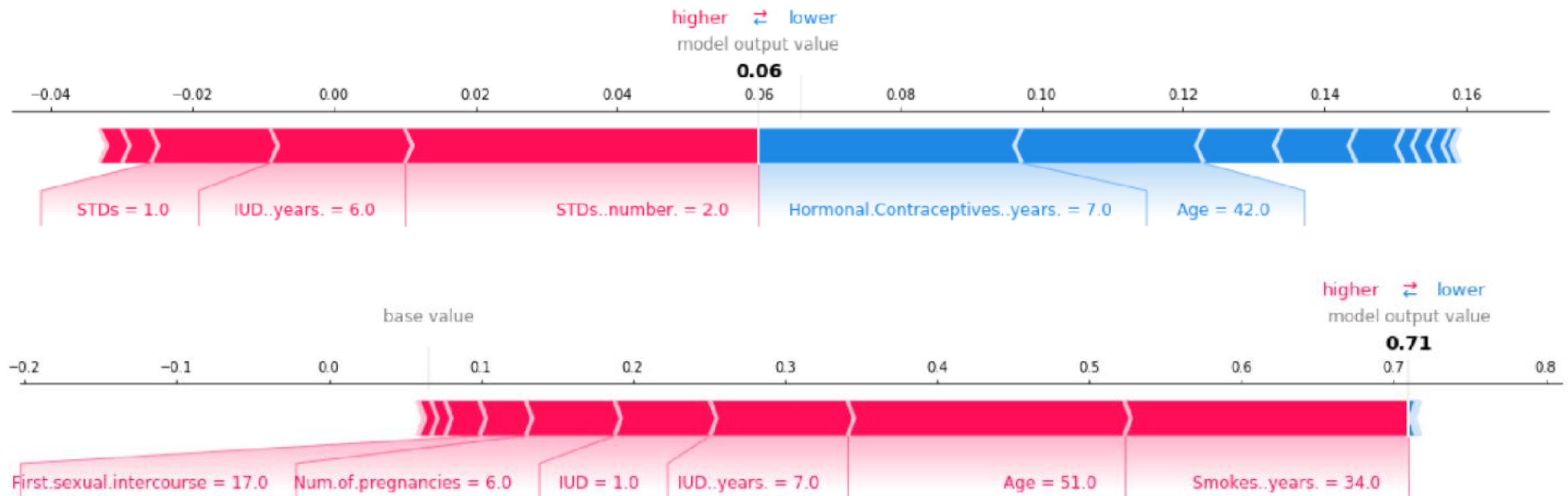
$$L(\hat{f}, g, \pi_x) = \sum_{z' \in Z} [\hat{f}(h_x(z')) - g(z')]^2 \pi_x(z')$$

SHapley Additive exPlanations (SHAP) (5)

- Comparison to LIME:
 - **Both SHAP and LIME** create “synthetic” samples (although in different ways) and use them to get a prediction from the BB model. This prediction then serves as an outcome for training a weighted linear regression, and the coefficients are used to interoperate the instance’s prediction.
 - **The big difference to LIME is the weighting** of the instances in the regression model:
 - **LIME weights the instances according to how close they are to the original instance**; Thinking about this as a coalition vector, the more 1’s (higher similarity compared to the original instance) => higher weight
 - **SHAP weights the sampled instances according to the weight the coalition would get in the Shapley value estimation**; Small coalitions (few 1’s) and large coalitions (i.e. many 1’s) get the largest weights.

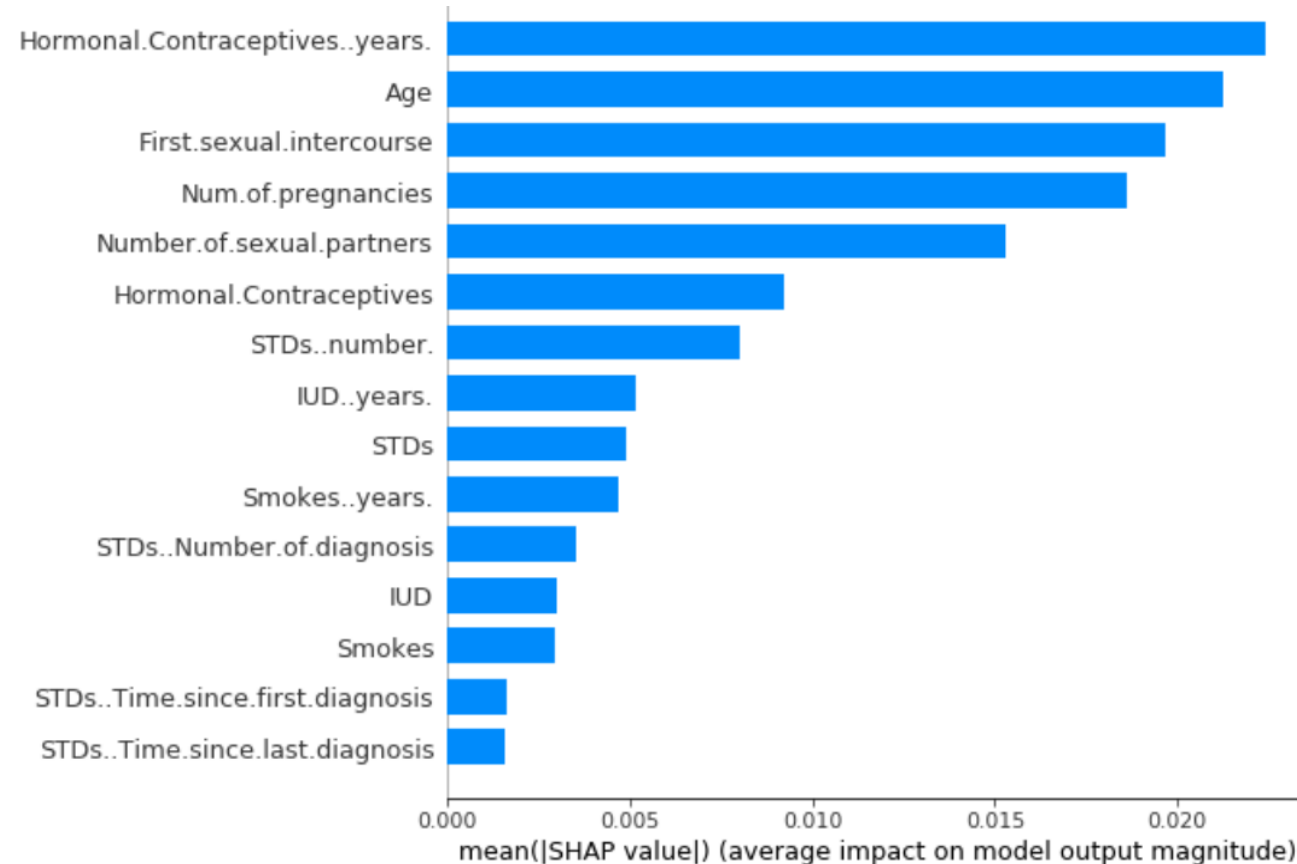
SHapley Additive exPlanations (SHAP) (6)

- Example – the following figure shows **SHAP explanation force plots** for two women from the cervical cancer dataset:



SHapley Additive exPlanations (SHAP) (7)

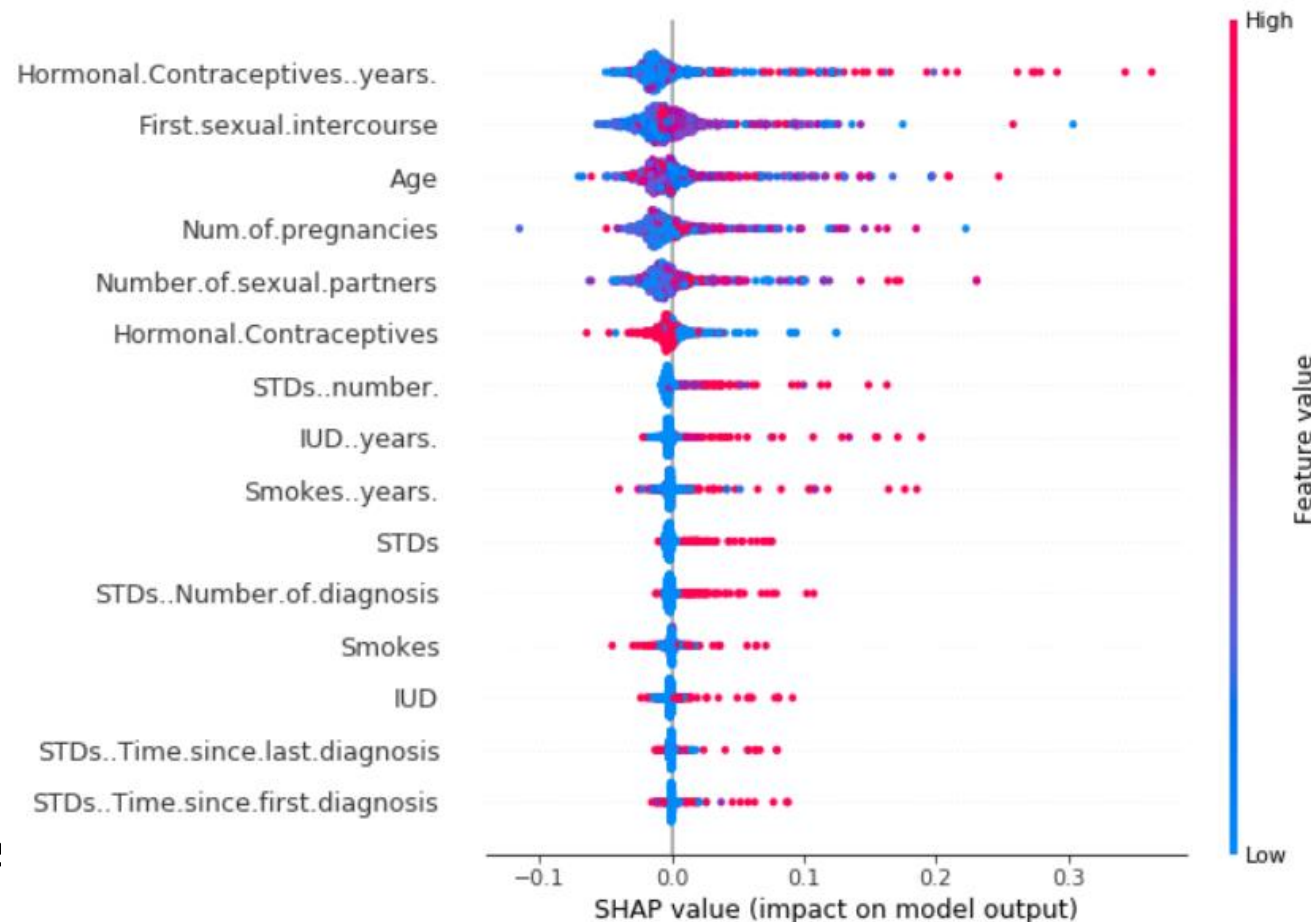
- Global Explainability:
 - Shapley values can be combined into global explanations.
 - **If we run SHAP for every instance, we get a matrix of Shapley values.** This matrix has one row per data instance and one column per feature.
 - **SHAP feature importance** average the absolute Shapley values per feature across the data:



SHapley Additive exPlanations (SHAP) (8)

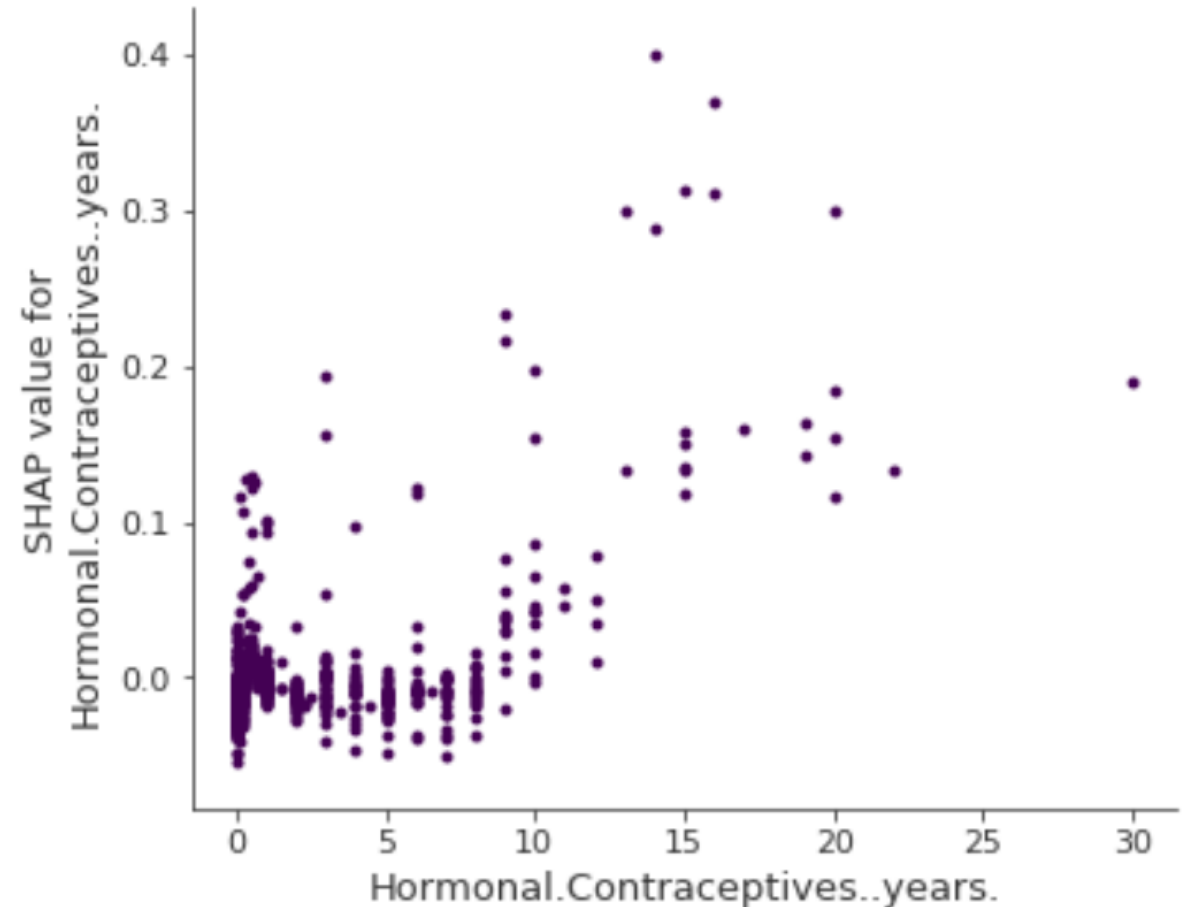
- SHAP Summary Plot:

- The summary plot **combines feature importance with feature effects.**
- Each point on the summary plot is a Shapley value for a feature and an instance
- **The position on the y-axis is determined by the global feature importance**
- **The position on the x-axis by the Shapley value. Overlapping points are jittered in y-axis direction, so we get a sense of the distribution of the Shapley values per feature.**
- **The color represents the value of the feature from low to high.**



SHapley Additive exPlanations (SHAP) (9)

- SHAP Dependence Plot
 - Allows to see the exact form of the relationship between the value of a feature and the impact on the prediction
 - An alternative to partial dependence plots. While PDP shows average effects, SHAP dependence also shows the variance on the y-axis. **Especially in case of interactions, this plot will be much more dispersed in the y-axis.**



SHapley Additive exPlanations (SHAP) (10)

- Advantages:
 - **All the advantages of Shapley values still apply**
 - SHAP has a **fast implementation for tree-based models**. This may have been the key to the popularity of SHAP, because the biggest barrier for adoption of Shapley values is the slow computation
 - SHAP **provides solutions both for local explainability and global explainability**, that are consistent with one another
- Disadvantages:
 - **KernelSHAP is slow**. This makes KernelSHAP impractical to use when you want to compute Shapley values for many instances.
 - KernelSHAP ignores feature dependence. Most other permutation-based interpretation methods also have this problem. TreeSHAP solves this problem by explicitly modeling the conditional expected prediction.

SHapley Additive exPlanations (SHAP) (11)

- Implementations:

- The authors implemented SHAP in the *shap* Python package. This implementation works for tree-based models in the scikit-learn library for Python.
- SHAP is integrated into the tree boosting frameworks xgboost and LightGBM.
- In R, there are the shapper and fastshap packages. SHAP is also included in the R xgboost package.

A few more things

The interpretability-performance tradeoff

- There is a tradeoff between model interpretability and performance
- If we want higher predictive performance, we can compromise on interpretability and use explainability methods to get intuition regarding model predictions

